

```
import { createClient } from 'src:@supabase/supabase-js@2';
// Signal Engine -- Structural Confluence + Zone Contract (spec v4.2, section 5-7 & 52-59)
// structural_v2. Ini adalah engine yang DIPANGGIL LANGSUNG oleh cron produksi lewat
// trigger_signal_pipeline_mtf() (job 65/66/67) -- perbaikan di sini WAJIB ada di sini,
// bukan cuma di function generate-signals biasa (yang tidak dipakai cron aktif).
//
// v4.2 update (dari structural_v2 versi v4.1): H1/H4 dihapus total dari engine (section
// 5.2/3.3) karena data intraday tidak reliable & tidak tersedia gratis dari IDX EOD.
// - Daily sekarang cuma evaluasi D1 (entry=D1, confirm=null, bias=D1) -- bukan lagi
// confluence H1->H4->D1. SL/TP/struktur semua dari D1 EOD.
// - Swing tetap D1 (entry & SL) -> W1 (bias & target), tidak berubah.
//
// Sisanya (naik dari versi mtf lama, buy-only, banyak fallback R:R fabrikasi):
// - Sekarang BUY *dan* SELL (bearish alert), bukan cuma BUY.
// - Support/Resistance sekarang ZONE (price_low/price_high/strength/touch_count/
// retest_count), bukan angka tunggal (section 54). Zone dipersist ke tabel
// structure_zones sebagai immutable snapshot per run (section 61).
// - TP1/TP2 WAJIB berasal dari zone struktural valid. Tidak ada lagi fallback R:R
// yang mengarah target (section 56.4). Kalau tidak ada zone TP1 valid -> NO SIGNAL.
// Kalau TP1 valid tapi tidak ada zone kedua -> tp2 = NULL + tp2_reason.
// - Overextension/chasing protection (section 58): BUY diblokir kalau harga sudah
// terlalu jauh dari support zone (formula versioned per tier).
// - entry_type (AREA/TRIGGER/RETEST/SUPPORT) dan trigger_state (breakout/breakdown
// state machine, section 55) dihitung dari histori candle, bukan cuma candle terakhir.
// - Setiap stock yang diproses (BUY/SELL/NO_SIGNAL/FAILED) dicatat ke
// signal_pipeline_events dengan reason eksplisit (section 52.1/23.3 observability).
//
// FIX (audit 17 Agustus 2026): recordEvent() SEBELUMNYA menulis stage='signal_decision'
// dan status='BUY'/SELL/'NO_SIGNAL' langsung ke signal_pipeline_events. Nilai itu
// melanggar CHECK constraint tabel (stage harus salah satu dari ingestion/quality/
// indicator/structure/confluence/signal/snapshot/notification; status harus OK/FAILED/
// SKIPPED) -- akibatnya SETIAP insert gagal secara diam-diam (error tidak pernah dicek)
// dan tabel observability ini selalu kosong walau fungsi jalan tiap hari. Sekarang
// outcome asli (BUY/SELL/NO_SIGNAL/FAILED) dipetakan ke status enum yang valid dan
// disimpan juga di detail.outcome supaya informasinya tidak hilang. Section 23.3 juga
// menuntut failure tidak boleh menyamar sebagai NO SIGNAL -- sekarang kegagalan teknis
// (batch fetch gagal, insert zone gagal, insert signal gagal) direkam eksplisit sebagai
// status FAILED, bukan cuma masuk counter debugSamples yang tidak persisten.
//
// FIX (audit 17 Agustus 2026, bag. 4): resolveDataSource() cuma mengenali source
// 'IDX_MANUAL' sebagai IDX asli. Padahal worker fetch-idx-eod (auto, dibuat 17 Agustus
// 2026) menulis candle dengan source 'IDX_AUTO' -- akibatnya walau fetch-idx-eod SUKSES
// dan candle hari ini benar-benar dari IDX, signal tetap ditandai data_source =
// 'YAHOO_FALLBACK' selamanya (salah, melanggar section 3.2/18.2 soal audit data_source).
// Sekarang IDX_MANUAL dan IDX_AUTO dua-duanya dianggap sumber IDX yang valid.
//
// FIX (20 Agustus 2026): getExpiry() hardcoded 15:30 WIB hari yang sama diganti dengan
// call ke DB function get_next_signal_expiry() yang baca market_calendar.
//
// Daily : D1 (entry, Stop Loss, bias & Target -- satu timeframe, tanpa confirm)
// Swing : D1 (entry & Stop Loss) -> W1 (bias & Target)
const CONCURRENCY = 20;
const FORMULA_VERSION = 'structural_v2';
const PIVOT_LEFT = 2;
const PIVOT_RIGHT = 2;
const CANDLE_LIMIT = 100;
const MIN_CANDLES = 30;
const SR_BUFFER_PCT = 0.003;
const ZONE_MERGE_TOLERANCE_PCT = 0.012;
const RETEST_TOLERANCE_PCT = 0.01;
const BREAKOUT_LOOKBACK_BARS = 8;
const PIPELINE_EVENT_CHUNK = 500;
const ZONE_INSERT_CHUNK = 300;
const OVEREXTENSION_MAX_PCT = {
  daily: 0.07,
  swing: 0.12
};
const TIERS = {
  daily: { tier: 'daily', entryTf: 'D1', confirmTf: null, biasTf: 'D1' },
  swing: { tier: 'swing', entryTf: 'D1', confirmTf: null, biasTf: 'W1' }
};
function findPivots(candles) {
  const highs = [];
  const lows = [];
  for (let i = PIVOT_LEFT; i < candles.length - PIVOT_RIGHT; i++) {
    const windowH = candles.slice(i - PIVOT_LEFT, i + PIVOT_RIGHT + 1).map((c) => c.high);
    const windowL = candles.slice(i - PIVOT_LEFT, i + PIVOT_RIGHT + 1).map((c) => c.low);
    if (candles[i].high === Math.max(...windowH)) highs.push({ idx: i, price: candles[i].high, candleId: candles[i].id });
    if (candles[i].low === Math.min(...windowL)) lows.push({ idx: i, price: candles[i].low, candleId: candles[i].id });
  }
  return { highs, lows };
}
function buildZones(pivots, zoneType, timeframe, totalBars) {
  if (pivots.length === 0) return [];
  const sorted = [...pivots].sort((a, b) => a.price - b.price);
  const clusters = [];
  let current = [sorted[0]];
  for (let i = 1; i < sorted.length; i++) {
    const clusterMean = current.reduce((s, p) => s + p.price, 0) / current.length;
    if (Math.abs(sorted[i].price - clusterMean) / clusterMean <= ZONE_MERGE_TOLERANCE_PCT) {
      current.push(sorted[i]);
    } else {
      clusters.push(current);
      current = [sorted[i]];
    }
  }
}
```

```

clusters.push(current);
return clusters.map((cluster) => {
const prices = cluster.map((p) => p.price);
const rawLow = Math.min(...prices);
const rawHigh = Math.max(...prices);
const buffer = Math.max(rawHigh - rawLow, rawHigh * SR_BUFFER_PCT) / 2;
const priceLow = rawLow - buffer;
const priceHigh = rawHigh + buffer;
const touchCount = cluster.length;
const retestCount = Math.max(0, touchCount - 1);
const oldestIdx = Math.min(...cluster.map((p) => p.idx));
const ageInBars = Math.max(0, totalBars - 1 - oldestIdx);
const strengthScore = touchCount * 2 + retestCount * 3 + Math.min(ageInBars, 50) / 10;
const strength = strengthScore >= 8 ? 'STRONG' : strengthScore >= 4 ? 'MODERATE' : 'WEAK';
return {
zone_type: zoneType, timeframe, price_low: priceLow, price_high: priceHigh,
mid_price: (priceLow + priceHigh) / 2, touch_count: touchCount, retest_count: retestCount,
strength_score: Number(strengthScore.toFixed(2)), strength, age_in_bars: ageInBars,
source_pivot_candle_ids: cluster.map(p => p.candleId)
};
});
}

function readStructure(candles, indicator, timeframe) {
if (candles.length < MIN_CANDLES || !indicator) return null;
const { highs, lows } = findPivots(candles);
const lastClose = candles[candles.length - 1].close;
let swingBias = null;

if (highs.length >= 2 && lows.length >= 2) {
const hh = highs[highs.length - 1].price > highs[highs.length - 2].price;
const hl = lows[lows.length - 1].price > lows[lows.length - 2].price;
const lh = highs[highs.length - 1].price < highs[highs.length - 2].price;
const ll = lows[lows.length - 1].price < lows[lows.length - 2].price;
if (hh && hl) swingBias = 'bullish';
else if (lh && ll) swingBias = 'bearish';
}

let emaBias = null;
if (indicator.ema21 != null && indicator.ema50 != null) {
if (lastClose > indicator.ema21 && indicator.ema21 > indicator.ema50) emaBias = 'bullish';
else if (lastClose < indicator.ema21 && indicator.ema21 < indicator.ema50) emaBias = 'bearish';
}

const bias = swingBias && emaBias && swingBias === emaBias ? swingBias : null;
return {
bias, lastClose,
supportZones: buildZones(lows, 'SUPPORT', timeframe, candles.length),
resistanceZones: buildZones(highs, 'RESISTANCE', timeframe, candles.length)
};
}

function nearestZoneBelow(zones, price) {
const below = zones.filter((z) => z.price_low <= price);
if (below.length === 0) return null;
return below.reduce((best, z) => z.price_high > best.price_high ? z : best);
}

function nearestZoneAbove(zones, price) {
const above = zones.filter((z) => z.price_high >= price);
if (above.length === 0) return null;
return above.reduce((best, z) => z.price_low < best.price_low ? z : best);
}

function secondZoneAbove(zones, first) {
const candidates = zones.filter((z) => z.price_low > first.price_high);
if (candidates.length === 0) return null;
return candidates.reduce((best, z) => z.price_low < best.price_low ? z : best);
}

function secondZoneBelow(zones, first) {
const candidates = zones.filter((z) => z.price_high < first.price_low);
if (candidates.length === 0) return null;
return candidates.reduce((best, z) => z.price_high > best.price_high ? z : best);
}

function classifyTrigger(candles, zone, kind) {
const window = candles.slice(-BREAKOUT_LOOKBACK_BARS);
if (window.length === 0) return null;
const last = window[window.length - 1];
if (kind === 'BREAKOUT') {
const breakoutIdx = window.findIndex((c) => c.close > zone.price_high);
if (breakoutIdx === -1) {
if (last.high >= zone.price_low && last.close <= zone.price_high) {
const distToEdge = (zone.price_high - last.close) / zone.price_high;
return distToEdge <= RETEST_TOLERANCE_PCT ? 'BREAKOUT_CANDIDATE' : 'INTRABAR_TOUCH';
}
}
return null;
}

const after = window.slice(breakoutIdx + 1);
const dippedBackIn = after.some((c) => c.low <= zone.price_high && c.low >= zone.price_low);
if (last.close <= zone.price_low) return 'FAILED_BREAKOUT';
if (dippedBackIn && last.close > zone.price_high) return 'BREAKOUT_RETEST';
return 'BREAKOUT_CONFIRMED';
} else {
const breakdownIdx = window.findIndex((c) => c.close < zone.price_low);
if (breakdownIdx === -1) {
if (last.low <= zone.price_high && last.close >= zone.price_low) {
const distToEdge = (last.close - zone.price_low) / zone.price_low;
return distToEdge <= RETEST_TOLERANCE_PCT ? 'BREAKDOWN_CANDIDATE' : 'INTRABAR_TOUCH';
}
}
return null;
}

const after = window.slice(breakdownIdx + 1);
const poppedBackIn = after.some((c) => c.high >= zone.price_low && c.high <= zone.price_high);

```

```

if (last.close >= zone.price_high) return 'FAILED_BREAKDOWN';
if (poppedBackIn && last.close < zone.price_low) return 'BREAKDOWN_RETEST';
return 'BREAKDOWN_CONFIRMED';
}
}

function classifyOverextension(entryPrice, supportZone, tier) {
if (!supportZone) return { status: 'UNKNOWN', distancePct: null };
const distancePct = (entryPrice - supportZone.price_high) / supportZone.price_high;
if (distancePct < 0) return { status: 'NORMAL', distancePct };
return { status: distancePct > OVEREXTENSION_MAX_PCT[tier] ? 'OVEREXTENDED' : 'NORMAL', distancePct };
}

function isSameWibDay(tslso, now) {
const wibOffsetMs = 7 * 60 * 60 * 1000;
const a = new Date(new Date(tslso).getTime() + wibOffsetMs);
const b = new Date(now.getTime() + wibOffsetMs);
return a.getUTCFullYear() === b.getUTCFullYear() && a.getUTCMonth() === b.getUTCMonth() && a.getUTCDate() === b.getUTCDate();
}

// FIX (20 Agustus 2026): sebelumnya hardcoded 15:30 WIB hari yang sama — signal
// langsung EXPIRED sebelum sempat ACTIVE. Sekarang pakai market_calendar via RPC.
async function getExpiry(supabase, tier, now) {
try {
const { data, error } = await supabase.rpc('get_next_signal_expiry', {
from_ts: now.toISOString(),
tier: tier === 'swing' ? 'swing' : 'daily',
});
if (error || !data) throw new Error(String(error?.message ?? 'no data'));
return data;
} catch (e) {
console.error('getExpiry fallback:', e);
const wibOffsetMs = 7 * 60 * 60 * 1000;
const wibNow = new Date(now.getTime() + wibOffsetMs);
const d = tier === 'swing' ? 2 : 1;
return new Date(Date.UTC(
wibNow.getUTCFullYear(), wibNow.getUTCMonth(), wibNow.getUTCDate() + d, 8, 50, 0
)).toISOString();
}
}

function resolveDataSource(entryCandles) {
const last = entryCandles[entryCandles.length - 1];
return last?.source === 'IDX_MANUAL' || last?.source === 'IDX_AUTO' ? 'IDX' : 'YAHOO_FALLBACK';
}

async function fetchTf(supabase, stockId, tf, now) {
const [{ data: candlesDesc, error: cErr }, { data: indRow, error: iErr }] = await Promise.all([
supabase.from('candles').select('id, ts, open, high, low, close, volume, source').eq('stock_id', stockId).eq('timeframe', tf).order('ts', { ascending: false }).limit(CANDLE_LIMIT),
supabase.from('indicators').select('ema5, ema9, ema21, ema50, rsi14, macd_line, macd_signal, stoch_k, stoch_d, volume_avg20').eq('stock_id', stockId).eq('timeframe', tf).maybeSingle()
]);
if (cErr) throw cErr;
if (iErr) throw iErr;
let candles = (candlesDesc ?? []).slice().reverse();
if ((tf === 'D1' || tf === 'W1') && candles.length > 0 && isSameWibDay(candles[candles.length - 1].ts, now)) {
candles = candles.slice(0, -1);
}
return { candles, indicator: indRow };
}

Deno.serve(async (req) => {
const supabase = createClient(Deno.env.get('SUPABASE_URL'), Deno.env.get('SUPABASE_SERVICE_ROLE_KEY'));
const url = new URL(req.url);
const tierParam = (url.searchParams.get('tier') ?? 'daily').toLowerCase();
const offset = Number(url.searchParams.get('offset') ?? '0');
const limit = Number(url.searchParams.get('limit') ?? '300');
if (tierParam !== 'daily' && tierParam !== 'swing') {
return new Response(JSON.stringify({ error: 'tier tidak didukung: ${tierParam}' }), { status: 400 });
}
const cfg = TIERS[tierParam];
const now = new Date();
const { data: jobRun, error: jobErr } = await supabase.from('job_runs').insert({
job_name: 'generate-signals-mtf-${cfg.tier}', status: 'RUNNING', started_at: now.toISOString()
}).select('id').single();
if (jobErr || !jobRun) {
return new Response(JSON.stringify({ error: jobErr?.message ?? 'failed to start job_run' }), { status: 500 });
}
const jobRunId = jobRun.id;
const { data: stocks, error } = await supabase.from('stocks').select('id, ticker').eq('is_active', true).order('ticker').range(offset, offset + limit - 1);
if (error || !stocks) {
await supabase.from('job_runs').update({ status: 'ERROR', finished_at: new Date().toISOString(), detail: { error: error?.message } }).eq('id', jobRunId);
return new Response(JSON.stringify({ error: error?.message ?? 'no stocks' }), { status: 500 });
}
let totalBuy = 0, totalSell = 0, totalNoConfluence = 0, totalSkipped = 0, totalFailed = 0;
let totalSetupInvalid = 0, totalOverextended = 0;
const debugSamples = [];
const pipelineEvents = [];
const recordEvent = (stockId, outcome, detail) => {
const status = outcome === 'FAILED' ? 'FAILED' : outcome === 'NO_SIGNAL' ? 'SKIPPED' : 'OK';
pipelineEvents.push({ stock_id: stockId, tier: cfg.tier, stage: 'signal', status, detail: { ...detail, outcome }, job_run_id: jobRunId });
};
for (let i = 0; i < stocks.length; i += CONCURRENCY) {
const batch = stocks.slice(i, i + CONCURRENCY);
const results = await Promise.allSettled(batch.map(async (s) => {
const entry = await fetchTf(supabase, s.id, cfg.entryTf, now);
const confirm = cfg.confirmTf ? await fetchTf(supabase, s.id, cfg.confirmTf, now) : null;
const bias = cfg.biasTf === cfg.entryTf ? entry : await fetchTf(supabase, s.id, cfg.biasTf, now);
return { stock: s, entry, confirm, bias };
})));
for (let ri = 0; ri < results.length; ri++) {
const r = results[ri];
const failedStockId = batch[ri]?.id;

```

```

if (r.status !== 'fulfilled') {
totalFailed++;
debugSamples[ `error-${totalFailed}` ] = String(r.reason);
if (failedStockId) recordEvent(failedStockId, 'FAILED', { reason: 'candle_or_indicator_fetch_failed', detail: String(r.reason) });
continue;
}
const { stock, entry, confirm, bias } = r.value;
const structEntry = readStructure(entry.candles, entry.indicator, cfg.entryTf);
const structConfirm = confirm ? readStructure(confirm.candles, confirm.indicator, cfg.confirmTf) : null;
const structBias = cfg.biasTf === cfg.entryTf ? structEntry : readStructure(bias.candles, bias.indicator, cfg.biasTf);
if (!structEntry || !structBias || (cfg.confirmTf && !structConfirm)) {
totalSkipped++;
recordEvent(stock.id, 'NO_SIGNAL', { reason: 'insufficient_data' });
continue;
}

const biases = [structEntry.bias, structBias.bias, ...(structConfirm ? [structConfirm.bias] : [])];
const allBullish = biases.every((b) => b === 'bullish');
const allBearish = biases.every((b) => b === 'bearish');
if (!allBullish && !allBearish) {
totalNoConfluence++;
recordEvent(stock.id, 'NO_SIGNAL', { reason: 'no_timeframe_confluence', biases });
continue;
}

const direction = allBullish ? 'BUY' : 'SELL';
const entryPrice = structEntry.lastClose;
const dataSource = resolveDataSource(entry.candles);
const zonesToPersist = [];
if (direction === 'BUY') {
const supportZone = nearestZoneBelow(structEntry.supportZones, entryPrice);
const resistanceZoneTp1 = nearestZoneAbove(structBias.resistanceZones, entryPrice);
if (!supportZone) { totalSetupInvalid++; recordEvent(stock.id, 'NO_SIGNAL', { reason: 'no_valid_support_zone', direction }); continue; }
if (!resistanceZoneTp1 || resistanceZoneTp1.mid_price <= entryPrice) { totalSetupInvalid++; recordEvent(stock.id, 'NO_SIGNAL', { reason: 'no_valid_resistance_zone_for_tp1', direction }); continue; }
const stopLoss = supportZone.price_low * (1 - SR_BUFFER_PCT);
if (stopLoss >= entryPrice) { totalSetupInvalid++; recordEvent(stock.id, 'NO_SIGNAL', { reason: 'invalid_stop_loss', direction }); continue; }
const overext = classifyOverextension(entryPrice, supportZone, cfg.tier);
if (overext.status !== 'NORMAL') { totalOverextended++; recordEvent(stock.id, 'NO_SIGNAL', { reason: overext.status === 'OVEREXTENDED' ? 'overextended' : 'unknown_overextension', distance_pct: overext.distancePct }); continue; }
const tp1 = resistanceZoneTp1.mid_price;
const tp2Zone = secondZoneAbove(structBias.resistanceZones, resistanceZoneTp1);
const tp2 = tp2Zone?.mid_price ?? null;
const tp2Reason = tp2Zone ? null : 'NO_VALID_SECOND_RESISTANCE';
const triggerState = classifyTrigger(entry.candles, supportZone, 'BREAKOUT');
const withinSupportZone = entryPrice >= supportZone.price_low && entryPrice <= supportZone.price_high * (1 + RETEST_TOLERANCE_PCT);
const entryType = withinSupportZone ? 'SUPPORT' : triggerState === 'BREAKOUT_RETEST' ? 'RETEST' : triggerState === 'BREAKOUT_CANDIDATE' ? 'TRIGGER' : 'AREA';
const risk = entryPrice - stopLoss;
const riskReward = risk > 0 ? Math.abs(tp1 - entryPrice) / risk : null;
zonesToPersist.push(supportZone, resistanceZoneTp1);
if (tp2Zone) zonesToPersist.push(tp2Zone);
const { data: oldActive } = await supabase.from('signals').select('id').eq('stock_id', stock.id).eq('signal_tier', cfg.tier).eq('status', 'ACTIVE').is('superseded_by', null);
const { data: zoneIds, error: zoneErr } = await insertZones(supabase, stock.id, zonesToPersist, now);
if (zoneErr) { totalFailed++; debugSamples[stock.ticker] = `zone_insert_failed`; recordEvent(stock.id, 'FAILED', { reason: 'zone_insert_failed', direction, detail: zoneErr }); continue; }
const { data: inserted, error: insErr } = await supabase.from('signals').insert({
stock_id: stock.id, timeframe: cfg.entryTf, signal_tier: cfg.tier,
entry_timeframe: cfg.entryTf, confirm_timeframe: cfg.confirmTf, bias_timeframe: cfg.biasTf,
direction, entry_price: entryPrice,
buy_area_low: supportZone.price_low, buy_area_high: entryPrice,
tp1, tp2, tp1_reason: null, tp2_reason: tp2Reason,
stop_loss: stopLoss, initial_stop_loss: stopLoss, risk_reward: riskReward,
support_level: supportZone.mid_price, resistance_level: resistanceZoneTp1.mid_price,
entry_type: entryType, trigger_state: triggerState, overextension_status: overext.status,
data_source: dataSource,
support_zone_id: zoneIds.get(zoneKey(supportZone)) ?? null,
resistance_zone_id: zoneIds.get(zoneKey(resistanceZoneTp1)) ?? null,
tp1_zone_id: zoneIds.get(zoneKey(resistanceZoneTp1)) ?? null,
tp2_zone_id: tp2Zone ? zoneIds.get(zoneKey(tp2Zone)) ?? null : null,
status: 'ACTIVE', formula_version: FORMULA_VERSION, engine_version: 'V1',
evidence: {
structure: { entry_bias: structEntry.bias, confirm_bias: structConfirm?.bias ?? null, bias_bias: structBias.bias, basis: 'swing_structure(HH-HL/LH-LL) + EMA21_vs_EMA50' },
support: zoneEvidence(supportZone), resistance: zoneEvidence(resistanceZoneTp1),
tp2_zone: tp2Zone ? zoneEvidence(tp2Zone) : null,
trigger: { entry_type: entryType, trigger_state: triggerState },
overextension: { status: overext.status, distance_pct: overext.distancePct, max_allowed_pct: OVEREXTENSION_MAX_PCT[cfg.tier] },
invalidation: { level: stopLoss },
timeframes: { entry: cfg.entryTf, confirm: cfg.confirmTf, bias: cfg.biasTf },
data_quality: { candle_count_entry: entry.candles.length, candle_count_bias: bias.candles.length, data_source: dataSource },
formula_version: FORMULA_VERSION, generated_at: now.toISOString()
},
triggered_at: now.toISOString(),
expires_at: await getExpiry(supabase, cfg.tier, now)
}).select('id').single();
if (insErr || !inserted) { totalFailed++; debugSamples[stock.ticker] = String(insErr?.message); recordEvent(stock.id, 'FAILED', { reason: 'signal_insert_failed', direction, detail: insErr?.message }); continue; }
if (oldActive && oldActive.length > 0) {
await supabase.from('signals').update({ status: 'INVALIDATED', superseded_by: inserted.id }).in('id', oldActive.map((o) => o.id));
}
totalBuy++;
recordEvent(stock.id, 'BUY', { entry_type: entryType, trigger_state: triggerState, tp2_reason: tp2Reason, data_source: dataSource });
} else {
const resistanceZone = nearestZoneAbove(structEntry.resistanceZones, entryPrice);
const supportZoneTp1 = nearestZoneBelow(structBias.supportZones, entryPrice);
if (!resistanceZone) { totalSetupInvalid++; recordEvent(stock.id, 'NO_SIGNAL', { reason: 'no_valid_resistance_zone', direction }); continue; }
if (!supportZoneTp1 || supportZoneTp1.mid_price >= entryPrice) { totalSetupInvalid++; recordEvent(stock.id, 'NO_SIGNAL', { reason: 'no_valid_support_zone_for_target', direction }); continue; }
const invalidation = resistanceZone.price_high * (1 + SR_BUFFER_PCT);
if (invalidation <= entryPrice) { totalSetupInvalid++; recordEvent(stock.id, 'NO_SIGNAL', { reason: 'invalid_invalidation_level', direction }); continue; }
const downsideSupport1 = supportZoneTp1.mid_price;
const support2Zone = secondZoneBelow(structBias.supportZones, supportZoneTp1);
const downsideSupport2 = support2Zone?.mid_price ?? null;
const tp2Reason = support2Zone ? null : 'NO_VALID_SECOND_SUPPORT';

```

```

const brokenSupportZone = structEntry.supportZones.filter((z) => z.price_low > entryPrice).reduce((best, z) => !best || z.price_low < best.price_low ? z : best, null);
const breakdownState = brokenSupportZone ? classifyTrigger(entry.candles, brokenSupportZone, 'BREAKDOWN') : null;
const resistanceTriggerState = classifyTrigger(entry.candles, resistanceZone, 'BREAKOUT');
const lastCandle = entry.candles[entry.candles.length - 1];
const vol20 = entry.indicator?.volume_avg20 ?? null;
let bearishType;
if (breakdownState === 'BREAKDOWN_CONFIRMED' || breakdownState === 'BREAKDOWN_RETEST') {
  bearishType = 'BEARISH_BREAKDOWN';
} else if (resistanceTriggerState === 'FAILED_BREAKOUT') {
  bearishType = 'BEARISH_REJECTION';
} else if (vol20 && lastCandle.volume && Number(lastCandle.volume) > vol20 * 1.5 && lastCandle.close < lastCandle.open) {
  bearishType = 'BEARISH_DISTRIBUTION_INDICATION';
} else {
  bearishType = 'BEARISH_CONTINUATION';
}
const risk = invalidation - entryPrice;
const riskReward = risk > 0 ? Math.abs(downsideSupport1 - entryPrice) / risk : null;
zonesToPersist.push(resistanceZone, supportZoneTp1);
if (support2Zone) zonesToPersist.push(support2Zone);
if (brokenSupportZone) zonesToPersist.push(brokenSupportZone);
const { data: oldActive } = await supabase.from('signals').select('id').eq('stock_id', stock.id).eq('signal_tier', cfg.tier).eq('status', 'ACTIVE').is('superseded_by', null);
const { data: zoneIds, error: zoneErr } = await insertZones(supabase, stock.id, zonesToPersist, now);
if (zoneErr) { totalFailed++; debugSamples[stock.ticker] = 'zone_insert_failed'; recordEvent(stock.id, 'FAILED', { reason: 'zone_insert_failed', direction, detail: zoneErr }); continue; }
const { data: inserted, error: insErr } = await supabase.from('signals').insert({
  stock_id: stock.id, timeframe: cfg.entryTf, signal_tier: cfg.tier,
  entry_timeframe: cfg.entryTf, confirm_timeframe: cfg.confirmTf, bias_timeframe: cfg.biasTf,
  direction, entry_price: entryPrice,
  tp1: downsideSupport1, tp2: downsideSupport2, tp1_reason: null, tp2_reason: tp2Reason,
  stop_loss: invalidation, initial_stop_loss: invalidation, risk_reward: riskReward,
  support_level: supportZoneTp1.mid_price, resistance_level: resistanceZone.mid_price,
  entry_type: null, trigger_state: breakdownState ?? resistanceTriggerState,
  overextension_status: null, data_source: dataSource,
  bearish_type: bearishType, bearish_trigger: resistanceZone.mid_price,
  invalidation_downside_support_1: downsideSupport1, downside_support_2: downsideSupport2,
  support_zone_id: supportZoneTp1 ? zoneIds.getKey(supportZoneTp1) ?? null : null,
  resistance_zone_id: zoneIds.getKey(resistanceZone) ?? null,
  tp1_zone_id: zoneIds.getKey(supportZoneTp1) ?? null,
  tp2_zone_id: support2Zone ? zoneIds.getKey(support2Zone) ?? null : null,
  status: 'ACTIVE', formula_version: FORMULA_VERSION, engine_version: 'v1',
  evidence: {
    structure: { entry_bias: structEntry.bias, confirm_bias: structConfirm?.bias ?? null, bias_bias: structBias.bias, basis: 'swing_structure(HH-HL/LH-LL) + EMA21_vs_EMA50' },
    support: zoneEvidence(supportZoneTp1), resistance: zoneEvidence(resistanceZone),
    tp2_zone: support2Zone ? zoneEvidence(support2Zone) : null,
    broken_support_zone: brokenSupportZone ? zoneEvidence(brokenSupportZone) : null,
    trigger: { type: bearishType, breakdown_state: breakdownState, resistance_trigger_state: resistanceTriggerState, level: resistanceZone.mid_price },
    invalidation: { level: invalidation },
    timeframes: { entry: cfg.entryTf, confirm: cfg.confirmTf, bias: cfg.biasTf },
    data_quality: { candle_count_entry: entry.candles.length, candle_count_bias: bias.candles.length, data_source: dataSource },
    formula_version: FORMULA_VERSION, generated_at: now.toISOString()
  },
  triggered_at: now.toISOString(),
  expires_at: await getExpiry(supabase, cfg.tier, now)
}).select('id').single();
if (insErr || !inserted) { totalFailed++; debugSamples[stock.ticker] = String(insErr?.message); recordEvent(stock.id, 'FAILED', { reason: 'signal_insert_failed', direction, detail: insErr?.message }); continue; }
if (oldActive && oldActive.length > 0) {
  await supabase.from('signals').update({ status: 'INVALIDATED', superseded_by: inserted.id }).in('id', oldActive.map(o => o.id));
}
totalSell++;
recordEvent(stock.id, 'SELL', { bearish_type: bearishType, breakdown_state: breakdownState, tp2_reason: tp2Reason, data_source: dataSource });
}
}
}

let pipelineEventError = null;
for (let i = 0; i < pipelineEvents.length; i += PIPELINE_EVENT_CHUNK) {
  const { error: peErr } = await supabase.from('signal_pipeline_events').insert(pipelineEvents.slice(i, i + PIPELINE_EVENT_CHUNK));
  if (peErr) { pipelineEventError = peErr.message; console.error('[generate-signals-mtf] gagal insert signal_pipeline_events:', peErr.message); }
}

await supabase.from('job_runs').update({
  status: 'SUCCESS', finished_at: new Date().toISOString(),
  detail: { buy: totalBuy, sell: totalSell, no_confluence: totalNoConfluence, setup_invalid: totalSetupInvalid, overextended: totalOverextended, skipped: totalSkipped, failed: totalFailed, pipeline_event_error: pipelineEventError }
}).eq('id', jobRunId);
return new Response(JSON.stringify({
  tier: cfg.tier, entry_timeframe: cfg.entryTf, confirm_timeframe: cfg.confirmTf, bias_timeframe: cfg.biasTf,
  offset, limit, total: stocks.length, formula_version: FORMULA_VERSION, job_run_id: jobRunId,
  buy: totalBuy, sell: totalSell, no_confluence: totalNoConfluence, setup_invalid: totalSetupInvalid,
  overextended: totalOverextended, skipped_insufficient_data: totalSkipped, failed: totalFailed,
  pipeline_events_recorded: pipelineEvents.length, pipeline_event_error: pipelineEventError, debug_samples: debugSamples
}), { headers: { 'Content-Type': 'application/json' } });
}

function zoneKey(z) {
  return `${z.zone_type}-${z.timeframe}-${z.price_low.toFixed(4)}-${z.price_high.toFixed(4)}`;
}

function zoneEvidence(z) {
  return { price_low: z.price_low, price_high: z.price_high, mid_price: z.mid_price, timeframe: z.timeframe, touch_count: z.touch_count, retest_count: z.retest_count, strength: z.strength, strength_score: z.strength_score, age_in_bars: z.age_in_bars };
}

async function insertZones(supabase, stockId, zones, now) {
  const uniqueByKey = new Map();
  for (const z of zones) uniqueByKey.set(zoneKey(z), z);
  const list = [...uniqueByKey.values()];
  if (list.length === 0) return { data: new Map(), error: null };
  const rows = list.map(z => ({
    stock_id: stockId, timeframe: z.timeframe, zone_type: z.zone_type,
    price_low: z.price_low, price_high: z.price_high, source_timeframe: z.timeframe,
    source_pivot_candle_ids: z.source_pivot_candle_ids, touch_count: z.touch_count,
    retest_count: z.retest_count, strength_score: z.strength_score, strength: z.strength,
    age_in_bars: z.age_in_bars, formula_version: FORMULA_VERSION, generated_at: now.toISOString()
  }));

```

```

    });
    const idMap = new Map();
    for (let i = 0; i < rows.length; i += ZONE_INSERT_CHUNK) {
      const chunk = rows.slice(i, i + ZONE_INSERT_CHUNK);
      const { data, error } = await supabase.from('structure_zones').insert(chunk).select('id, zone_type, timeframe, price_low, price_high');
      if (error) return { data: idMap, error: error.message };
      for (const row of data ?? []) {
        idMap.set(`${row.zone_type}-${row.timeframe}-${Number(row.price_low).toFixed(4)}-${Number(row.price_high).toFixed(4)}`, row.id);
      }
    }
    return { data: idMap, error: null };
  }
}
```